



SECURITY STANDARDS & CHECKLIST

Project Security Checklist

A universal, do-this / avoid-that reference for any project — websites, web apps, APIs, databases, mobile back-ends, and single-file HTML/JS tools. Covers client, server, backend, frontend, secrets, git, SQL, auth, RLS, TLS, and everything in between.

How to use it: Hand this document to Claude at the start of any project and say “apply this checklist.” Claude will work through the relevant sections for whatever you’re building and flag anything that needs attention. For quick experiments, the **Minimum Set** on the next page is the non-negotiable core.

Before you start

Security is layered: a weakness in any one place can undo the rest. Work top-to-bottom for anything real or public. For a throwaway experiment, at least cover the Minimum Set below.

★ The Minimum Set — never skip these

If you do nothing else, do these. They stop the most common and most damaging mistakes.

- ☐ No secrets in code or git — use env vars and a .gitignore.
- ☐ Only public keys in the browser; secret/service keys stay server-side.
- ☐ Parameterized queries everywhere — never concatenate SQL.
- ☐ Hash passwords (bcrypt/argon2) or use a trusted auth provider.
- ☐ Enforce authorization on the server; enable RLS on every table.
- ☐ HTTPS everywhere; HttpOnly + Secure + SameSite cookies.
- ☐ Validate all input server-side; escape all output.
- ☐ Scan the repo for leaked secrets before going public.

☐ Action to take / verify (tick it off) ☒ Never do this / red flag

01 Secrets, API Keys & Credentials

Anything that grants access is a secret. Treat it like one.

- ☐ Keep every secret (API keys, DB passwords, tokens, private keys, webhook signing secrets) out of source code, HTML, and JS.
 - ☐ Load secrets from environment variables or a secret manager (Vault, AWS/GCP Secrets Manager, Doppler, 1Password) — never from committed files.
 - ☐ Use different keys for dev, staging, and production; never reuse a prod key locally.
 - ☐ Know which keys are PUBLIC (publishable / anon / client) vs SECRET (service_role / server / admin). Only public keys may reach the browser.
 - ☐ Restrict every key you can: allowed domains/referrers, IP ranges, scopes, and spend limits.
 - ☐ Rotate keys on a schedule and immediately on any suspected leak or staff departure.
 - ☐ Provide a committed .env.example listing variable NAMES only (no values) so the project is reproducible.
-
- ☒ Never hardcode a secret “just for now” — it will be committed and forgotten.
 - ☒ Never put a secret (service_role, private API key) in front-end code or a single-file HTML app. The browser is public.
 - ☒ Never send secrets over email, chat, or screenshots.

02 Git & Version Control — What to Upload, What Not

The repo is forever. Assume anything you push could become public.

- ☐ COMMIT: source code, configuration WITHOUT secrets, .env.example, dependency lockfiles, README, .gitignore, migrations, tests.
 - ☐ Add a .gitignore BEFORE the first commit (see Appendix A) and confirm it covers secrets, build output, and dependencies.
 - ☐ Install a secret-scanning pre-commit hook (gitleaks, git-secrets, or trufflehog) so leaks are blocked before they are pushed.
 - ☐ Scan the whole history — not just the latest commit — for secrets (see Appendix B).
 - ☐ Protect the main branch: require pull-request review, block force-push, and enable branch protection.
 - ☐ Default new repositories to PRIVATE; only make public deliberately after a secret scan.
 - ☐ Consider signed commits (GPG/SSH) for provenance on shared or public repos.
-
- ✗ DO NOT UPLOAD: .env / .env.* files, credentials, *.pem / *.key / certificates, service-account JSON, node_modules or venv, build artifacts (dist/, .next/), database dumps or backups, log files, .DS_Store / editor folders.
 - ✗ Never put secrets, tokens, or customer data in commit messages, issues, or PR descriptions.
 - ✗ If a secret is ever committed: ROTATE it first (assume it is compromised), THEN scrub history with git filter-repo or BFG. Deleting the file alone does not remove it from history.

03 Authentication (Login & Passwords)

Prove who the user is — without ever storing a readable password.

- ☐ Store passwords only as salted hashes using bcrypt, scrypt, or Argon2id — never plaintext, MD5, or SHA-1.
 - ☐ Prefer a vetted auth provider/library (Supabase Auth, Auth0, Clerk, Firebase Auth, Django/Rails auth) over rolling your own.
 - ☐ Allow long passphrases; check against known-breached password lists; do not force awkward composition rules.
 - ☐ Offer and encourage MFA / 2FA for all accounts, mandatory for admins.
 - ☐ Rate-limit and lock out after repeated failed logins; add CAPTCHA on abuse.
 - ☐ Make password reset use single-use, short-expiry tokens; invalidate them after use.
 - ☐ Return generic auth errors (“invalid email or password”) so you don’t reveal which accounts exist.
 - ☐ Verify email ownership before granting sensitive capabilities.
-
- ✗ Never email or display a user’s actual password.
 - ✗ Never build your own crypto or password hashing from scratch.
 - ✗ Never trust a login state that lives only on the client.

04 Sessions, Tokens & Cookies*A stolen session is a stolen account. Keep them short and sealed.*

- ☐ Set auth cookies with HttpOnly, Secure, and SameSite (Lax or Strict).
- ☐ Use short-lived access tokens plus rotating refresh tokens.
- ☐ Invalidate sessions/tokens on logout, password change, and role change.
- ☐ For JWTs: verify the signature, and validate exp, iss, and aud on every request; pin the expected algorithm.
- ☐ Regenerate the session ID on login to prevent session fixation.
- ☐ Set idle and absolute session timeouts appropriate to the app's sensitivity.

- ✗ Never store long-lived or highly-privileged tokens in localStorage/sessionStorage — they are exposed to any XSS. Prefer HttpOnly cookies.
- ✗ Never accept a JWT with “alg: none” or a client-chosen algorithm.
- ✗ Never put passwords or sensitive PII inside a JWT payload — it is only base64, not encrypted.

05 Authorization & Access Control (incl. RLS)*Authentication says who you are; authorization says what you may touch. Enforce it on the server.*

- ☐ Enforce authorization on the SERVER for every request — client-side checks are UX only, never security.
- ☐ Default to DENY; grant the minimum access each role needs (least privilege).
- ☐ Enable Row Level Security (RLS) on EVERY table in Postgres/Supabase and write explicit per-role policies; a table with RLS off is fully open to anyone holding the anon key.
- ☐ Test RLS with each role (anon, authenticated user, admin) and confirm users cannot read or write other users' rows.
- ☐ Prevent IDOR: on every object access, verify the current user owns or is allowed that specific record — don't trust an ID from the request.
- ☐ Separate roles clearly (anon, user, admin, service) and re-check permission on every sensitive action, not just at login.
- ☐ Protect and audit admin routes and destructive actions.

- ✗ Never rely on a hidden UI element or a disabled button as an access control.
- ✗ Never expose the service_role / admin key to code that a user can reach.
- ✗ Never assume “if they got the URL/ID they must be allowed”.

06 Database & SQL Injection (“SQL-Free”)*Never let user input become part of a query's structure.*

- ☐ Use parameterized queries / prepared statements / ORM bindings for 100% of queries that touch user input.
- ☐ For parts that can't be parameterized (table/column names, ORDER BY, LIMIT direction), validate against a strict allow-list.
- ☐ Give the application a least-privilege DB user — no superuser, only the rights it actually needs.
- ☐ Enable RLS and encrypt sensitive columns; encrypt data at rest.
- ☐ Keep the database off the public internet; restrict access by IP/VPC and require TLS.
- ☐ Automate backups AND periodically test a restore; store backups encrypted.
- ☐ Apply the same discipline to NoSQL and search queries — they have injection risks too (validate objects, avoid passing raw user input as query operators).

-
- × Never build SQL by concatenating or string-interpolating user input.
 - × Never expose raw database errors to the client.
 - × Never ship an app connecting to the DB as an admin/root account.

07 Input Validation & Output Encoding*Distrust all input. Encode all output for where it lands.*

- ☐ Validate every input on the SERVER: type, length, range, format, and allowed values. Client validation is for UX only.
- ☐ Prefer allow-lists (what IS permitted) over block-lists (what isn't).
- ☐ Encode/escape output for its exact context: HTML body, HTML attribute, JavaScript, URL, CSS, or SQL.
- ☐ Enforce maximum sizes on request bodies, fields, arrays, and uploads.
- ☐ Guard against XML external entities (XXE), unsafe deserialization, and server-side template injection.
- ☐ Normalize and canonicalize input (encoding, paths) before validating to defeat bypasses.

-
- × Never trust a value because the front-end “already validated” it.
 - × Never reflect raw user input back into a page or response unescaped.
 - × Never pass user input into shell commands, file paths, or eval without strict validation.

08 Frontend / Client-Side Security

Everything shipped to the browser is public and editable. Design around that.

- ☐ Assume all HTML, JS, and network calls are visible and modifiable by the user — put no secret and no trust in the client.
 - ☐ Prevent XSS: render user data with `textContent` / framework escaping; sanitize any user-supplied HTML with `DOMPurify` before inserting.
 - ☐ Set a strong Content-Security-Policy to block inline and unexpected scripts.
 - ☐ Add Subresource Integrity (SRI) hashes to third-party CDN scripts and styles.
 - ☐ Avoid `eval`, `new Function`, and inline event handlers; keep logic in reviewed code.
 - ☐ Strip debug info, verbose errors, internal IDs, and source maps from production builds.
 - ☐ Single-file HTML/JS apps: keep zero API secrets in the file; call a thin backend or serverless proxy that holds the key and validates requests.
-
- ✗ Never use `innerHTML` (or `dangerouslySetInnerHTML`) with unsanitized user data.
 - ✗ Never embed a private/secret API key in client JavaScript — view-source reveals it instantly.
 - ✗ Never enforce pricing, limits, or permissions only in the browser.

09 API Security

Every endpoint is a door. Authenticate, authorize, validate, and limit each one.

- ☐ Authenticate every endpoint except those intentionally public; authorize per resource (see IDOR in §05).
 - ☐ Rate-limit and throttle by user/key/IP; set quotas to blunt abuse and cost attacks.
 - ☐ Validate all path params, query strings, headers, and bodies against a schema.
 - ☐ Serve APIs over HTTPS only.
 - ☐ Configure CORS explicitly: list exact allowed origins; never combine wildcard origin with credentials.
 - ☐ Return only the fields the client needs; guard against mass-assignment / over-posting on writes.
 - ☐ Return consistent, minimal error responses — no stack traces or internal identifiers.
 - ☐ Put a gateway / WAF in front of public APIs where feasible.
-
- ✗ Never expose an admin or internal endpoint without auth because “nobody knows the URL”.
 - ✗ Never trust the client to send the correct `user_id` / `role` / `price` — derive it server-side from the session.
 - ✗ Never set `Access-Control-Allow-Origin: *` on authenticated endpoints.

10 Transport Security (SSL / TLS / HTTPS)*Encrypt everything in transit — internal traffic included.*

- ☐ Serve all traffic over HTTPS and redirect HTTP → HTTPS.
- ☐ Send HSTS (Strict-Transport-Security) so browsers refuse to downgrade.
- ☐ Use valid, auto-renewing certificates (e.g. Let's Encrypt); monitor expiry.
- ☐ Require TLS 1.2+; disable SSLv3/TLS 1.0/1.1 and weak ciphers.
- ☐ Set the Secure flag on cookies so they never travel over plain HTTP.
- ☐ Encrypt service-to-service and database connections, not just the public edge.

- ✗ Never send credentials, tokens, or PII over plain HTTP.
- ✗ Never disable certificate validation in clients/SDKs to “make it work”.
- ✗ Never mix HTTP assets into an HTTPS page (mixed content).

11 Security HTTP Headers*A few response headers stop whole classes of attacks. Set them once.*

- ☐ Content-Security-Policy — restrict where scripts, styles, and connections may come from.
- ☐ Strict-Transport-Security — enforce HTTPS (max-age, includeSubDomains).
- ☐ X-Content-Type-Options: nosniff — stop MIME sniffing.
- ☐ X-Frame-Options: DENY or CSP frame-ancestors — prevent clickjacking.
- ☐ Referrer-Policy: strict-origin-when-cross-origin (or tighter).
- ☐ Permissions-Policy — disable camera, mic, geolocation, etc. unless needed.
- ☐ Remove or mask Server / X-Powered-By version banners.

- ✗ Never ship a production site with default framework headers and no CSP.
- ✗ Never allow your pages to be framed by untrusted origins.

12 URLs, Redirects, CSRF & SSRF*URLs get logged, cached, and shared. Requests can be forged — in both directions.*

- ☐ Keep secrets, tokens, and PII OUT of URLs and query strings (they land in logs, history, and referrer headers).
- ☐ Validate and allow-list redirect targets to prevent open redirects.
- ☐ Use POST (not GET) for state-changing actions and protect them with anti-CSRF tokens or SameSite cookies.
- ☐ Prevent SSRF: validate and allow-list any URL your server fetches; block private IP ranges and cloud metadata endpoints (169.254.169.254).
- ☐ Use signed, expiring URLs for temporary access to private resources.

- ✗ Never pass a password-reset or session token as a plain URL parameter that gets logged.
- ✗ Never redirect the user to a URL taken verbatim from user input.
- ✗ Never let the server fetch an arbitrary user-supplied URL without validation.

13 Dependencies & Supply Chain

Most of your code is other people's code. Track it.

- ☐ Pin versions and commit lockfiles (package-lock.json, yarn.lock, poetry.lock, etc.).
 - ☐ Run automated dependency audits (npm audit, pip-audit, Dependabot, Snyk) and act on high/critical findings.
 - ☐ Update dependencies regularly; watch advisories/CVEs for what you use.
 - ☐ Minimize dependencies; vet a package's maintenance, popularity, and permissions before adding it.
 - ☐ Watch for typosquatting — confirm exact package names.
 - ☐ Review or disable untrusted install/postinstall scripts.
-
- ✗ Never blindly upgrade or auto-merge dependency changes into production without tests.
 - ✗ Never install packages from untrusted sources or random URLs.

14 File Uploads & Storage

An upload is untrusted input that persists. Contain it.

- ☐ Validate type, extension, and size — and verify the real content type via magic bytes, not just the filename.
 - ☐ Store uploads outside the web root or in object storage (S3, Supabase Storage), never in an executable path.
 - ☐ Generate random server-side filenames; never trust the client filename.
 - ☐ Serve downloads with the correct Content-Type and Content-Disposition; scan for malware where possible.
 - ☐ Lock down bucket/object permissions (private by default) and use signed URLs for access.
-
- ✗ Never allow uploaded files to be executed by the server.
 - ✗ Never leave a storage bucket public-by-default.
 - ✗ Never trust a file's extension or client-provided MIME type as proof of its content.

15 Error Handling, Logging & Monitoring

Log enough to investigate — never enough to leak.

- ☐ Show users generic error messages; keep stack traces and internals server-side only.
 - ☐ Log security-relevant events: logins, failures, permission changes, admin actions.
 - ☐ Centralize logs and alert on anomalies (spikes in errors, failed logins, rate-limit hits).
 - ☐ Keep audit trails for sensitive/destructive actions.
 - ☐ Set log retention and access controls; protect logs like data.
-
- ✗ Never log passwords, tokens, full card/PII, or secrets — redact them.
 - ✗ Never return internal exception details, SQL, or file paths to the client.

16 Configuration, Hosting & Deployment

Most breaches are misconfigurations, not exotic exploits.

- ☐ Separate dev / staging / prod fully; never point dev at prod data or secrets.
- ☐ Turn OFF debug mode and verbose errors in production.
- ☐ Pull secrets from a secret manager or the host's encrypted env store — not from files on disk.
- ☐ Apply least privilege to every service account, IAM role, and API token.
- ☐ Keep OS, runtimes, and containers patched; remove unused services and default accounts.
- ☐ Back up data and configuration; test disaster recovery.
- ☐ Restrict who can access production and require MFA for it.

- ✗ Never deploy with debug endpoints, default credentials, or sample admin accounts enabled.
- ✗ Never expose dashboards, admin panels, or databases directly to the public internet.

17 CI/CD & Build Pipeline

The pipeline holds the keys to production. Guard it.

- ☐ Store CI/CD secrets in the platform's encrypted secret store; mask them in logs.
- ☐ Run secret scanning and SAST/dependency scans in the pipeline; fail the build on critical issues.
- ☐ Give deploy credentials the least privilege needed and scope them per environment.
- ☐ Pin and review third-party CI actions/plugins by version or SHA.
- ☐ Require review and protect the pipeline configuration itself from unauthorized edits.

- ✗ Never echo secrets into build logs or artifacts.
- ✗ Never let a pull request from a fork run with access to production secrets.

18 Data Privacy & Compliance

The safest data is the data you never collected.

- ☐ Collect the minimum data you truly need; delete what you no longer use.
- ☐ Encrypt PII in transit and at rest; tightly control who can access it.
- ☐ Define data retention and deletion policies and honor deletion/export requests.
- ☐ Get consent and publish a privacy policy where laws (GDPR, CCPA, etc.) apply.
- ☐ Anonymize or pseudonymize data for analytics and testing.
- ☐ Vet third parties (analytics, payment, AI APIs) for what data they receive.

- ✗ Never use real customer PII in dev, test, or demo environments.
- ✗ Never send more personal data to a third-party service than the feature requires.

19 Pre-Launch Security Review — How to Check*Before anything goes live, run this pass. Verify, don't assume.*

- ☐ Scan the full repo AND git history for secrets (gitleaks / trufflehog).
- ☐ Run a dependency audit and resolve high/critical vulnerabilities.
- ☐ Test authorization: log in as user A and try to read/modify user B's data (IDOR); hit endpoints while logged out.
- ☐ Test inputs with injection and XSS payloads; try oversized and malformed input.
- ☐ Verify RLS policies with anon, user, and admin roles — confirm cross-user access fails.
- ☐ Check headers at securityheaders.com and TLS at sslabs.com/sslltest.
- ☐ Confirm production has debug OFF, no verbose errors, and no source maps leaking internals.
- ☐ Confirm .env and secrets are not deployed to any public path; view-source the client for stray keys.
- ☐ For serious apps, run an automated scan (OWASP ZAP) and consider a professional penetration test.

20 If Something Leaks — Incident Response*Speed limits the damage. Rotate first, investigate second.*

- ☐ Rotate/revoke the exposed credential immediately — assume it is already compromised.
- ☐ Invalidate affected sessions and tokens.
- ☐ Assess the blast radius from logs: what could the key access, and was it used?
- ☐ If it was committed to git, scrub it from history after rotating.
- ☐ Patch the root cause and add a check (scanner, test, review rule) so it can't recur.
- ☐ If personal data was exposed, follow your legal/notification obligations.

Appendix A — .gitignore Starter

Drop this in before your first commit. It keeps secrets, dependencies, build output, and data out of the repo. Adjust for your stack.

```
.gitignore
```

```
# ---- Secrets & credentials ----
.env
.env.*
!.env.example
*.pem
*.key
*.p12
*.pfx
*-service-account*.json
credentials.json
secrets.*

# ---- Dependencies ----
node_modules/
.venv/
venv/
__pycache__/_
vendor/

# ---- Build output ----
dist/
build/
.next/
out/
*.log

# ---- Local / OS / editor ----
.DS_Store
Thumbs.db
.idea/
.vscode/
*.local

# ---- Data & backups ----
*.sqlite
*.sql
*.dump
backup*/
```

Commit a `.env.example` with variable names only, so others can reproduce the setup without your secrets.

Appendix B — How to Check (Scan Commands)

Run these before going public and wire the secret scanners into a pre-commit hook and CI.

Scan whole git history for secrets	<code>gitleaks detect --source . -v</code>
Deep secret scan (filesystem + history)	<code>trufflehog git file://. --only-verified</code>
Block secrets before every commit	<code>gitleaks protect --staged # run from a pre-commit hook</code>
Audit Node dependencies	<code>npm audit --audit-level=high</code>
Audit Python dependencies	<code>pip-audit</code>
List what a commit will actually upload	<code>git status --short && git ls-files</code>
Check if <code>.env</code> is tracked (should print nothing)	<code>git ls-files grep -E '\.env\$'</code>

Appendix C — Single-File HTML / JS App: The Short List

When the whole app is one HTML/JS file the user can read and edit, the rules narrow to these.

- ☐ No API keys or secrets anywhere in the file — view-source shows everything.
- ☐ Any privileged call goes through a small backend/serverless proxy that holds the key and checks the request.
- ☐ User content is inserted with `textContent` or sanitized (DOMPurify) — never raw `innerHTML`.
- ☐ External scripts loaded over HTTPS with Subresource Integrity (SRI) hashes.
- ☐ No `eval` / `new Function` on user input; a `Content-Security-Policy` meta tag is set.
- ☐ Treat all logic and “limits” as advisory — the user can edit the file; enforce anything real on a server.
- ☐ Don't store other people's data in a purely client-side app without a real backend and access control.

This checklist is a practical baseline, not a guarantee. For high-risk or regulated systems, add a professional security review or penetration test.

· Give a copy to Claude on any project to have it applied for you.